Color Diary

Micro Project Report

Submitted to the APJ Abdul Kalam University, Kerala
for the partial fulfilment of the requirements for the award of the degree

Bachelor of Technology
in
Computer Science and Engineering
by:

CELIA VICTOR - TOC24CS057
HIBA FATHIMA A S - TOC24CS083
LEMA FATHIMA - TOC24CS106
HYFA KH - TOC24CS085
GOUTHAMI M K - TOC24CS080
GAYATHRI DEEPU - TOC24CS077

OCTOBER 2025

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Toc H INSTITUTE OF SCIENCE & TECHNOLOGY TIST

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
Toc H INSTITUTE OF SCIENCE AND TECHNOLOGY
Arakkunnam, Ernakulam, Kerala – 682 313

CERTIFICATE

This is to certify that the microproject work report entitled 'COLOR DIARY' submitted by **CELIA VICTOR (TOC24CS057), GAYATHRI DEEPU (TOC22CS077), GOUTHAMI M K (TOC25CS080), HIBA FATHIMA A S (TOC24CS083), HYFA K H (TOC24CS085) & LEMA FATHIMA (TOC24CS106)** during the third semester, is a bonafide record of the microproject for the subject PBCST304 – Object Oriented Programming, prescribed by APJ Abdul Kalam Technological University, Kerala, carried out under our guidance and supervision, during the academic year 2025 - '26.

Ms.Mima Manual
Project Coordinator
Assistant Professor, CSE
TIST

Dr.SreelaSreedhar
HOD
Dept.of CSE
TIST

DECLARATION

We hereby declare that the microproject report **Color Diary** submitted for the partial fulfilment of the requirements of the Third Semester Microproject for the subject **PBCST304 – Object Oriented Programming**, prescribed by APJ Abdul Kalam Technological University, Kerala is a bonafide work done by us under the guidance of Asst. Prof. Mima Manual.

The submission represents our ideas in our own words and where ideas or words of others have not been included. We have adequately and accurately cited and referred to the original sources.

We also declare that we have adhered to the ethics and integrity and have not misinterpreted or fabricated any data or idea or fact or source in our submission .We understand that any violation of the above will be a case for disciplinary action by the institution and/or the University and can also evoke penal actions from the sources which have thus not been properly cited or from whom proper permission has not been obtained .This report has not previously formed the basis for the award of any degree, diploma or similar title of any other University.

Place: Arakkunnam

Date: 18 - 10 - 25

AS

CELIA VICTOR

HIBA FATHIMA

LEMA FATHIMA

HYFA K H

GOUTHAMI M K

GAYATHRI DEEPU

I

ABSTRACT

Color Diary is an interactive color-coded digital calendar designed to help users track and visualize their emotional wellbeing. Each calendar day is shaded in a color representing the user's mood rating, ranging from bright red for low moods to deep green for highly positive days. Users can personalize their experience by choosing from multiple color palettes such as Heatmap, Ocean, or Candy which offers visually distinct representations of mood variation throughout the month. The system also includes a login feature, allowing each user to securely access their personal diary and maintain privacy. Once logged in, the interface opens to an intuitive calendar view where users can easily navigate between months and years to reflect on past emotional patterns.

Clicking on any day opens a dedicated entry window where users can record their mood rating (on a scale of 1–10), select an expressive emoji, and write a brief diary note about their experiences. All entries are stored in an integrated database, implemented in SQLite. The application also offers a statistics dashboard that presents analytical insights such as monthly mood averages, trend graphs, and emotional fluctuations over time. Additionally, a weather window allows users to view real-time local weather information (temperature, rainfall, and conditions) providing some context that might influence mood. All these features together result in a digital diary that encourages mindfulness, emotional awareness, and self-reflection in a visually engaging and data-driven format.

CONTENTS

I Abstract **4**

II List of Figures **6**

III List of Tables **6**

1. Introduction

- **1.1 Background** **7**
- **1.2 Objectives of the Project** **8**
- **1.4 Relevance** **9**

2. System Requirement Specification (SRS)

- **2.1 Functional Requirements** **10**
- **2.2 Non-Functional Requirements** **13**
- **2.3 System Requirements (Hardware & Software)** **15**

3. System Design

- **3.1 Class Diagram** **17**
- **3.2 Database Design** **20**

4. Implementation & Results

- **4.1 Project Implementation Details** **23**
- **4.2 Test Cases** **27**
- **4.3 Sample Input and Output Screenshots** **29**

5. Conclusion

- **5.1 Summary of Work** **33**
- **5.2 Achievements and Observations** **33**
- **5.3 Limitations** **34**
- **5.4 Future Enhancements** **35**
- **5.5 Conclusion** **36**

IV References **37**

V Appendix

- **Source Code (Java + SQL snippets)** **38**

II LIST OF FIGURES

3.1 Class Diagram
4.1 Splash screen
4.2 Login Screen
4.3 Calendar view (Heatmap Theme)
4.4 Mood Entry Window
4.5 All Palettes
4.6 Statistics Window
4.7 Weather Window
Appendix.1 Sql table
Appendix.2 Source Code Snippet

III LIST OF TABLES

4.1 Test Cases Table

Chapter 1
INTRODUCTION

1.1 Background

In our day-to-day lives, emotions play a major role in how we think, act, and make decisions. However, most of us rarely take the time to notice how our feelings change from day to day. We may remember a few good or bad days but often forget the overall pattern. This is where the idea for *Color Diary* came from. The project began as a simple thought : what if a calendar could show how we felt each day using just colors instead of long diary entries or complicated charts? Over time, this small concept grew into a complete program that combines color psychology, data tracking, and user interaction.

Color Diary is a personal digital diary system that turns daily moods into colors. Instead of writing lengthy notes, users can record their emotional state using a number between 1 and 10. Each number corresponds to a color : lower values show cooler, darker shades like red and orange, while higher values appear as bright greens and yellows. By looking at the calendar, users can immediately understand how their month has been emotionally, without having to read through text. This idea makes mood tracking not only simpler but also visually engaging.

The project also focuses on making this experience technically strong. It uses Java Swing for the graphical interface, SQLite for the database, and JFreeChart for visualizing mood trends. These technologies were chosen because they are reliable, portable, and lightweight, which means the application can run on most computers without needing special configuration. As the project developed, new ideas were added, like integrating a weather window to show how the environment might affect mood, and allowing users to log in securely with their own account. What started as a creative experiment slowly became a fully functional application that combines emotion, color, and technology in a meaningful way.

1.2 Objectives of the project

The main objective of Color Diary is to create an easy and interactive way for users to track their moods and visualize emotional changes over time. The focus is not only on storing data but also on presenting it in a way that is simple to understand. The system uses colors to make emotional data more relatable and easy to remember. Through this project, we wanted to design a tool that encourages self-reflection without feeling heavy or clinical.

The specific goals of the project can be summarized as follows:

- To provide users with a digital diary where they can record daily moods using a rating scale from 1 to 10.
- To enable adding short notes and expressive emojis to make the entries more personal.
- To display the calendar view with each day colored according to the mood rating for a clear visual summary.
- To store user data in a local SQLite database so it remains accessible even without the internet.
- To include statistical analysis through charts and summaries that show trends in emotional wellbeing.
- To implement a login system that ensures data privacy for each user.
- To integrate a weather window that fetches real-time temperature, wind speed, and rainfall information.
- To maintain a lightweight, offline-capable design suitable for both students and general users.

Overall, the goal is to help users reflect on their own emotional patterns in a non-intrusive way. The project demonstrates how simple ideas like combining mood, color, and data can come together to create a meaningful experience using core programming concepts.

1.3 Relevance

In the present world, mental wellness and emotional awareness have become increasingly important. People today are looking for tools that can help them understand their feelings and habits better. Color Diary addresses this need in a light and approachable way. Unlike professional medical applications, it is not meant to diagnose or treat, but to encourage users to check in with themselves regularly. By visually representing emotions, it allows users to observe long-term patterns that might otherwise go unnoticed.

Another important aspect of this project is the way it connects emotions with real-life factors. The weather feature, for example, adds an environmental context to the diary. Users can notice whether rainy days make them feel down or if bright days improve their mood. This connection between emotion and nature makes the diary more than just a record — it becomes a reflection tool that helps users understand how external factors influence them.

From a technical perspective, the project is also relevant to software engineering education. It brings together several key topics from object-oriented programming: encapsulation, modularity, event-driven design, GUI development, and database connectivity. Each class in the program has a clear role and interacts smoothly with others, which shows good programming practice. For students and beginners, this project provides a practical example of how to design, implement, and test a complete application that balances usability, logic, and visual design.

CHAPTER 2
System Requirement Specifications (SRS)

The System Requirement Specification defines what the *Color Diary* application is expected to do, how it should behave, and under what conditions it should perform its tasks. This section helps in understanding both the functional and non-functional expectations of the system before implementation. It acts as a foundation for design and development, ensuring that all features are properly planned, justified, and achievable within the project's scope.

The SRS also helps maintain clarity between different stages of the software lifecycle. For example, developers can use it as a checklist to confirm that all expected features are working, while future maintainers can refer to it when adding new improvements or debugging issues. Since this project is meant to be standalone, lightweight, and usable on most computers, the requirements are written with simplicity and practicality in mind.

2.1 Functional Requirements

Functional requirements describe what the system must do. They define the major functions, user interactions, and data-related tasks that make up the *Color Diary* experience. Every listed function plays an important role in making the diary not only useful but also easy to operate.

1. **User Login and Authentication**
    - The system must allow users to create an account with a username and password.
    - Login credentials are verified from the database before access is granted.
    - Passwords are securely stored in encrypted form to protect user privacy.
    - A logout option should be provided to safely exit the application.
2. **Mood Entry and Data Storage**
    - Users can record their daily mood by selecting a rating between 1 and 10.

- Each rating corresponds to a color code which will appear on the calendar.
- In addition to the numeric mood value, users can also add an emoji and a short note.
- All entries are saved in the SQLite database along with the date and user ID.
- If an entry already exists for a particular date, it should be updated instead of duplicated.

3. **Calendar View and Navigation**
   - The main interface displays a monthly calendar where each day is represented as a color-coded button.
   - Users can move between months using navigation arrows.
   - The current date is highlighted with a blue border for easy identification.
   - The interface must refresh automatically after a mood entry is saved.
4. **Color Palettes and Custom Themes**
   - Users can switch between different visual themes or color palettes.
   - Each palette represents the range of moods using distinct color transitions (for example, *Heatmap*, *Ocean*, *Candy*, etc.).
   - This feature adds personalization and allows the diary to reflect the user's aesthetic preference.
5. **Statistics and Analysis**
   - The system should generate statistical summaries for the current month, including:
     - Total number of mood entries recorded.

        - Average mood rating for the month.
        - A visual mood trend graph using JFreeChart.
    - The chart provides a quick overview of emotional variation throughout the month.

6. **Weather Integration**
    - Users can open a separate *Weather Window* to view live weather information for their location.
    - The app uses the **Open-Meteo API**, a free web service that provides current temperature, wind speed, and rainfall data.
    - The city name is entered by the user, and the program automatically fetches geographic coordinates and displays forecast details.
    - This feature helps users observe possible links between weather and mood.
7. **Offline Capability and Data Persistence**
    - Since the project uses SQLite, users can record and view entries even without an internet connection.
    - The weather window is the only feature that requires online access.
    - All diary data is stored locally and automatically loaded each time the program runs.
8. **User Interface Interaction**
    - The system must display clean, organized windows for all operations.
    - Dialog boxes are used for mood entry, while separate windows display weather and statistics.

- Each button must have clear labels and perform the expected function without lag or confusion.

These functions ensure that the application meets its main goal: allowing users to record, view, and analyze their moods in a way that is personal, meaningful, and visually appealing.

2.2 Non-Functional Requirements

Non-functional requirements describe how the system should perform its functions rather than what the functions themselves are. They focus on quality attributes like usability, performance, security, and design constraints.

1. **Usability**

    - The interface must be simple and intuitive so that users with no technical background can use it easily.

    - Labels, buttons, and menus should be clearly visible, with consistent layout and color schemes.

    - The system should provide quick feedback, such as confirmation messages after saving entries.

2. **Performance**

    - The application should load within a few seconds and respond to user actions without delay.

    - Database queries must be optimized for speed and reliability, even with larger datasets.

    - The overall performance must not degrade significantly as the number of diary entries increases.

3. **Reliability and Data Integrity**

    - Mood entries must be stored safely and retrieved correctly every time.

- SQLite provides a robust file-based storage mechanism that ensures consistency even in case of sudden program closure.
- The system should prevent duplicate or conflicting entries for the same date and user.

4. **Security**
    - Passwords must be hashed before being stored in the database.
    - User-specific data should only be accessible after successful login authentication.
    - No sensitive data should be transmitted in plain text.
5. **Portability**
    - The application must run on any system with Java installed, regardless of operating system.
    - Since SQLite is file-based, no additional database server configuration is required.
    - The program should function properly across Windows, macOS, and Linux.
6. **Maintainability and Modularity**
    - The program follows object-oriented design with each class handling a specific part of the functionality (e.g., MoodManager, DatabaseManager, WeatherWindow).
    - This modular design makes debugging and future feature additions easier.
    - Changes in one part of the program should not affect unrelated modules.
7. **User Experience**

- The interface colors and fonts are chosen to be visually comfortable and not distracting.
- Modern GUI conventions are used so that users can navigate naturally without needing instructions.
- Pop-up messages and alerts are minimal and only used when necessary.

8. **Scalability**
    - Though designed for personal use, the program's structure allows expansion into a multi-user or online version in the future.
    - The use of standard libraries (Swing, SQLite, HTTPClient) ensures that scaling or migration to larger databases like MySQL or PostgreSQL will be straightforward.

2.3 System Requirements (Hardware & Software)

This section lists the hardware and software needed for the development and smooth operation of *Color Diary*.

Hardware Requirements

- **Processor:** Intel i3 or equivalent and above
- **RAM:** Minimum 4 GB (8 GB recommended for development)
- **Storage:** At least 500 MB free space
- **Display:** 1366×768 resolution or higher for full UI visibility
- **Internet:** Required only for weather data retrieval

Software Requirements

- **Operating System:** Windows 10 or higher / Ubuntu 20.04+ / macOS Catalina+

- **Programming Language:** Java (JDK 17 or newer recommended)
- **Database:** SQLite (replacing MySQL for simplicity and offline capability)
- **Libraries Used:**
    - **Swing** – for graphical user interface
    - **JFreeChart** – for data visualization and charting
    - **org.json** – for parsing API responses
    - **java.net.http.HttpClient** – for connecting to online weather API
- **Build Tool:** Maven
- **IDE:** Eclipse or IntelliJ IDEA

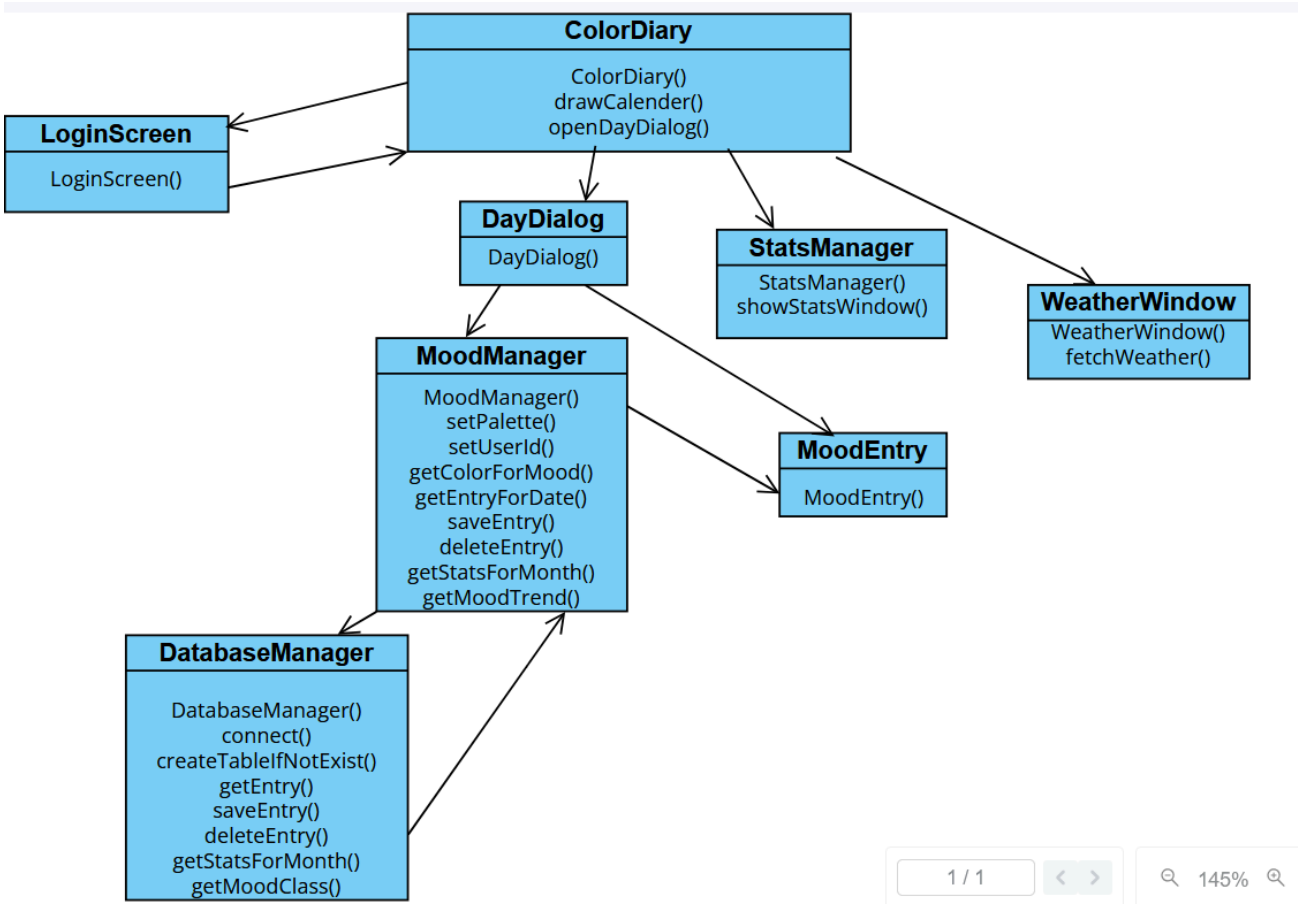
The combination of Java, Swing, and SQLite ensures that *Color Diary* remains lightweight and accessible without requiring complex setup. The program's environment is designed for easy replication, so other users or developers can install and run it simply by cloning the repository and executing the compiled JAR file.

CHAPTER 3
System design

3.1 Class Diagram

A class diagram is one of the most important representations in object-oriented software design. It visually shows the classes used in the system and the relationships between them, such as associations, dependencies, and compositions. In *Color Diary*, the main classes include:

- **ColorDiary** – The central class that handles the calendar display and user interactions.
- **LoginScreen** – Manages user login and authentication.
- **MoodManager** – Handles all mood-related logic such as saving, retrieving, and assigning colors to moods.
- **MoodEntry** – Represents a single record of a user's mood, emoji, and note for a particular date.
- **DatabaseManager** – Establishes connection with the SQLite database and performs queries.
- **DayDialog** – The window that allows the user to input daily mood entries.
- **StatsManager** – Generates statistical reports and mood trend charts.
- **WeatherWindow** – Displays real-time weather information using an online API.



Class Diagram Figure

This diagram shows how each class plays its role in a connected yet modular structure. The arrows represent dependencies, indicating which classes rely on others for data or functionality.

Overview of Class Interactions

When the program runs, it begins with the **ColorDiary** class, which serves as the main entry point of the application. From here, a splash screen appears briefly before transferring control to the **LoginScreen**, where users must enter their credentials. Once authentication is successful, the system creates a database connection through the **DatabaseManager**, which then provides access to the diary data.

The **ColorDiary** interface presents a navigable calendar that shows the current month's days as color-coded buttons. When a user clicks a date, the **DayDialog** class opens, allowing the user to record their mood, choose an emoji, and write a note. These inputs are processed and stored by the **MoodManager**, which uses the **DatabaseManager**

internally to perform database operations. The **MoodEntry** class acts as a data container for each diary record, ensuring that mood information is easily accessible and transferable between classes.

The **StatsManager** class is used to open a separate statistics window that displays a summary of the user's moods for the current month. It uses the *JFreeChart* library to plot line graphs showing mood variations over time. This visualization helps the user notice trends and patterns in emotional behavior.

The **WeatherWindow** operates independently but can be launched from the main interface. It connects to the **Open-Meteo API**, retrieves live weather data for a user-specified city, and displays information such as temperature, wind speed, and forecast. This feature makes the diary more interactive by linking mood and weather patterns, a relationship often discussed in psychology and wellness studies.

Class Relationships

Below is a textual explanation of the relationships between the classes:

1. **ColorDiary → LoginScreen**
   The ColorDiary class is launched after successful login. LoginScreen authenticates the user and then calls ColorDiary to display the main interface.

2. **ColorDiary → DatabaseManager**
   The main application uses DatabaseManager to connect to and communicate with the SQLite database.

3. **ColorDiary → MoodManager → DatabaseManager**
   When the user saves or retrieves mood entries, ColorDiary calls methods in MoodManager. MoodManager in turn interacts with DatabaseManager to execute SQL queries.

4. **ColorDiary → DayDialog → MoodManager**
   DayDialog handles user input for each day, such as the mood rating and notes. It passes this data to MoodManager for storage and retrieval.

5. **ColorDiary → StatsManager**
   When the user clicks the "Statistics" button, the StatsManager class is launched to calculate and display average mood and other statistics.

6. **ColorDiary → WeatherWindow**
   This relationship is optional. The user can open WeatherWindow independently

to view live weather updates without affecting diary data.

7. **MoodManager → MoodEntry**
   Each mood entry is stored as a MoodEntry object. This makes it easier to handle structured data rather than dealing directly with raw SQL results.

3.2 Database Design

Database design is a critical part of the system because it ensures that all diary entries, user accounts, and related data are stored efficiently and can be retrieved quickly when needed. For *Color Diary*, the database is kept simple to allow smooth offline operation and easy installation.

The system originally used **MySQL**, but it has since been migrated to **SQLite** for better portability and simpler setup. SQLite stores all data in a single file without requiring a separate server, which suits the offline, personal-use nature of the project.

Database Schema

The database consists of the following key tables:

1. users
This table stores user account information such as login credentials and profile data.

```
CREATE TABLE users (
    user_id INTEGER PRIMARY KEY AUTOINCREMENT,
    username TEXT NOT NULL UNIQUE,
    password_hash TEXT NOT NULL,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

2. mood_entries

This table records the user's daily mood ratings and related information.

```
CREATE TABLE mood_entries (
    entry_id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id INTEGER NOT NULL,
    entry_date DATE NOT NULL,
    mood_rating INTEGER,
    emoji TEXT,
    text_entry TEXT,
    FOREIGN KEY (user_id) REFERENCES users (user_id)
);
```

3. **events** *(planned feature)*

This optional table will allow users to add personal events, reminders, or holidays that appear on the calendar.

```
CREATE TABLE events (
    event_id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id INTEGER,
    event_date DATE,
    title TEXT,
    description TEXT,
    FOREIGN KEY (user_id) REFERENCES users (user_id)
);
```

Entity Relationships

- A **user** can have multiple **mood_entries**, establishing a one-to-many relationship.
- Each **mood_entry** belongs to one **user** only.

- In future updates, **events** will also be linked to users in the same way.

This structure ensures efficient organization of user data, maintains privacy between users, and allows simple SQL queries to retrieve information for monthly summaries or statistics.

Data Flow

1. When the user logs in, their credentials are checked in the users table.

2. Upon entering a mood rating, the data is saved into the mood_entries table through the MoodManager class.

3. When statistics are requested, the program retrieves average mood data from the same table and sends it to the StatsManager for graph plotting.

4. The weather data, being external, is not stored in the database but fetched dynamically using the **Open-Meteo API**.

Database Design Justification

The database design is deliberately minimal but fully functional. It uses normalized tables to prevent redundancy while still being easy to manage. SQLite's file-based nature eliminates the need for installation of a separate database server, which makes the project ideal for students and individual users. The design ensures data safety, scalability for future features (like events or reminders), and simplicity in backup and transfer.

CHAPTER 4

Implementation and Results

4.1 Project Implementation Details

The implementation stage of *Color Diary* focused on converting the system design into a working Java application. The coding was done using **Eclipse IDE**, and the project used **Maven** as the build tool to handle dependencies and organize the code structure neatly. The project follows an object-oriented design with clear separation between GUI, database logic, and data processing.

The main packages and files of the project are organized under the com.colordiary package. Each class handles one part of the overall program, making the structure modular and easier to debug or expand in the future. This modular design also allows new developers to quickly understand what each part of the system does.

Main Classes Implemented

1. ColorDiary.java

This is the **main application window** and the central point where all user interactions occur after logging in. It displays the monthly calendar, which uses color-coded buttons to represent each day's mood. The calendar is designed with a GridLayout, which neatly arranges the days in a 7-column grid format resembling a real-world calendar.

Each button's color is determined by the user's stored mood rating, using a gradient defined by the selected color palette. For instance, in the default "Heatmap" theme, red indicates a bad day while green shows a great day. Users can switch between different palettes like "Ocean," "Serenity," and "Candy," depending on their preference.

The ColorDiary class also contains navigation buttons (< and >) to move between months, a statistics button to open the StatsManager, and a dropdown menu to change the current color palette. When a user clicks on any specific date, a pop-up window (DayDialog) opens, allowing mood entry for that day.

2. LoginScreen.java

Before the diary can be accessed, the user must log in. The LoginScreen handles authentication and registration. This ensures that each user's data is stored separately in the database.

The class presents a small, clean window with fields for username and password, along with “Login” and “Create Account” buttons. When a new user signs up, their password is encrypted using a hash function before being saved to the database. This prevents unauthorized access and keeps data secure. Once credentials are verified, the system opens the main ColorDiary window and connects to the database for that specific user.

3. MoodManager.java

The MoodManager is responsible for everything related to moods: storing, retrieving, and assigning colors to each rating. This class interacts directly with the database using DatabaseManager.

It includes methods like:

- saveEntry(LocalDate date, int rating, String emoji, String notes)
- getEntryForDate(LocalDate date)
- getColorForMood(int rating)

Each mood entry is represented as an object of the MoodEntry class. The color assignment logic is written using switch-case structures for each palette. The palettes are designed to match emotional tones. for instance, “Serenity” moves from dark blue to green, symbolizing calm to joy, while “Mystic Fire” transitions from deep red to blue.

4. MoodEntry.java

MoodEntry acts as a **data model** for storing daily diary entries. It holds three main attributes (mood rating, emoji, and notes) along with the date associated with the entry. Instead of storing raw SQL results, the application creates instances of MoodEntry, making it easier to pass structured data between different parts of the program.

This approach follows good object-oriented design practice, ensuring clean data encapsulation.

5. DatabaseManager.java

DatabaseManager handles all database operations such as connecting to SQLite, executing SQL queries, and creating required tables. Originally, the system used MySQL, but it was later switched to SQLite to make the program portable and offline-friendly.

SQLite was integrated using the JDBC driver. When the application runs for the first time, it checks if the necessary tables (like users and mood_entries) exist. If not, it creates

them automatically. This makes the program self-contained. Users don't need to set up a separate database manually.

The main methods include:

- connect()
- createTables()
- insertMoodEntry()
- getMoodEntriesForMonth()
- verifyUser()

6. DayDialog.java

This class manages the **daily input window** where users record their mood for the day. It appears when the user clicks on a calendar day in ColorDiary.

The interface includes:

- A text field for entering a mood rating (1–10)
- A text field for choosing or typing an emoji
- A emoji button that opens an emoji picker dropdown
- A large text area for typing notes
- "Save" and "Cancel" buttons

When the user clicks **Save**, the input data is passed to the MoodManager class, which saves it to the database. If the same date already has an entry, the record is updated instead of duplicated.

This class uses Swing components like JDialog, JTextArea, and JPopupMenu. The window design is simple but clean, with balanced spacing and a responsive layout that fits all resolutions.

7. StatsManager.java

This class generates the **statistics view** for the user's recorded moods. It opens in a new window when the "Statistics" button is clicked in ColorDiary.

Using the **JFreeChart** library, it visualizes data through a line chart showing mood trends across the month. It also displays numerical summaries such as:

- Average mood score
- Total number of diary entries
- Highest and lowest rated days

These visual and numeric summaries help users reflect on their emotional patterns and see improvements over time. Future plans include adding comparisons across multiple months and showing "happiest" and "saddest" weeks.

8. WeatherWindow.java

The WeatherWindow provides an additional feature that fetches and displays live weather updates for a given city. It uses the **Open-Meteo API**, a free web service that returns weather data in JSON format.

The class sends an HTTP request to the API, retrieves the weather information, and displays it neatly in a JTextArea. The displayed data includes:

- Current temperature
- Wind speed
- Forecast for the coming days

This window helps users see how external factors like weather might correlate with mood, supporting the diary's emotional wellness goals.

4.2 Test Cases

Testing was carried out in small phases after each module was completed. The testing approach mainly followed manual functional testing, verifying that every button and input field worked as expected.

Below are some of the test cases used:

Test Case	Description	Expected Result	Actual Result
1	Login with valid credentials	Access granted	Passed
2	Login with wrong password	Access denied	Passed
3	Save new mood entry	Entry saved in DB	Passed
4	Update existing entry	Entry updated	Passed
5	Delete entry	Entry removed from DB	Passed
6	Change color palette	Calendar colors updated	Passed
7	Open statistics window	Graphs display correctly	Passed
8	View weather	Weather window opens	Passed

Testing confirmed that the application runs without major functional bugs. Only minor visual adjustments were needed in layout spacing.

Implementation Challenges

A few challenges were encountered during implementation:

- Integrating **JFreeChart** required adding external Maven dependencies.
- Handling the **SQLite connection** across multiple classes needed careful management of database resources to prevent leaks.
- Making the GUI consistent across different systems required adjusting LayoutManager settings.

- The emoji picker implementation was tricky due to Unicode handling in Java Swing.

Despite these challenges, the system was successfully implemented with all core features working as intended.

Results

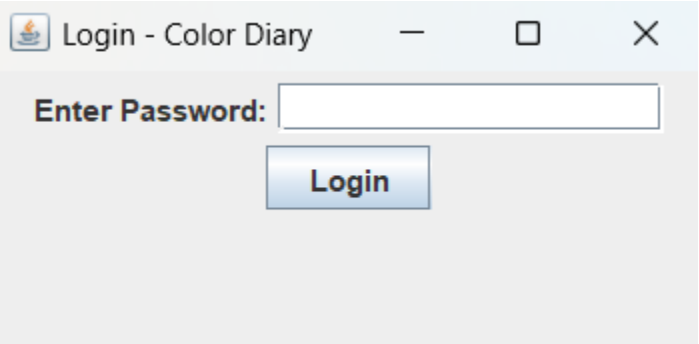
After several rounds of testing and refinement, the final *Color Diary* application met all the planned objectives. The program successfully lets users:

- Record and track moods daily
- Visualize their emotional data through colors and graphs
- Fetch real-time weather updates
- Switch between multiple visual themes
- Log in securely and access personal data

The app runs smoothly without requiring an internet connection for the main features, though weather updates need connectivity. The SQLite-based database ensures that data is stored locally and reliably. The modular design also means the app can be easily extended with new features in the future.

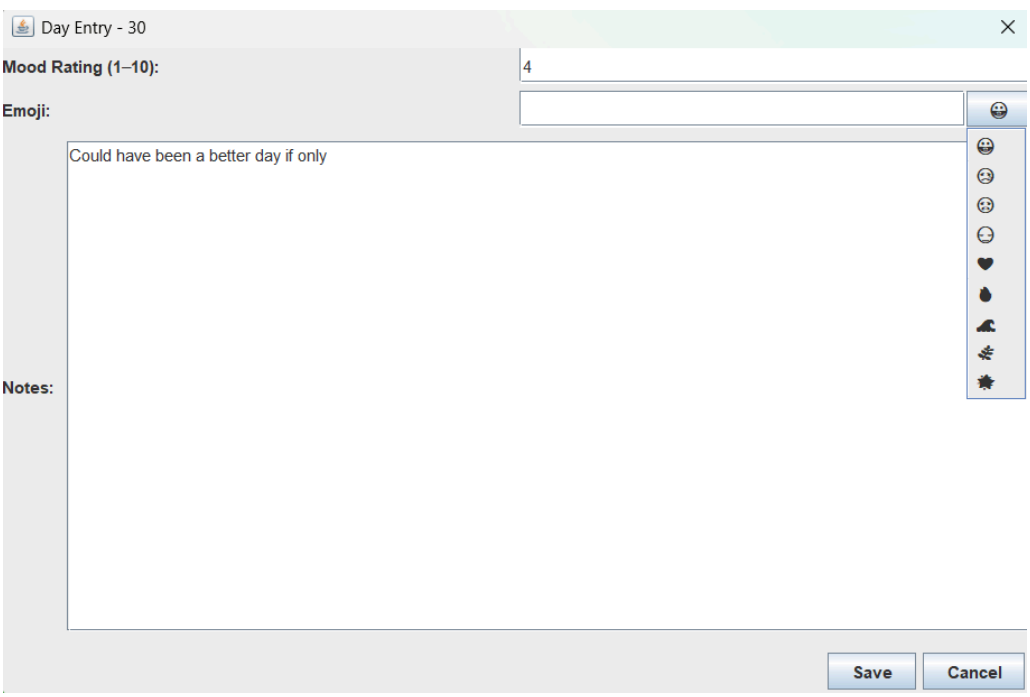
4.3 Sample Input and Output Screenshots

- Splash screen – “Color Diary” welcome

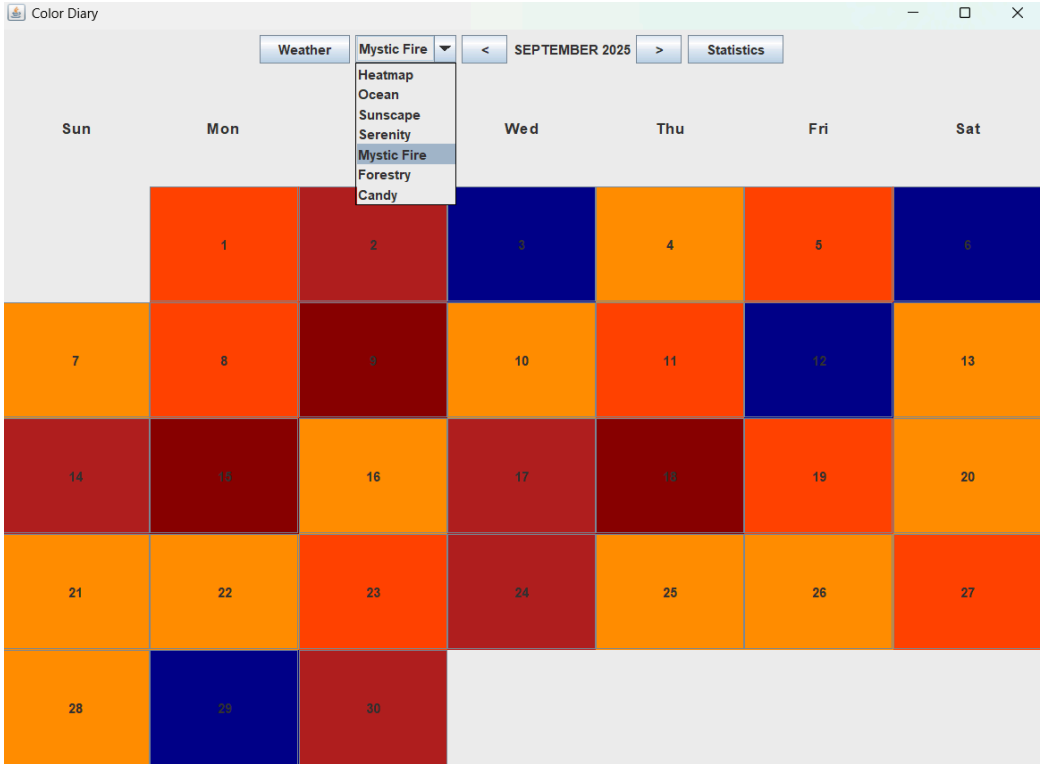


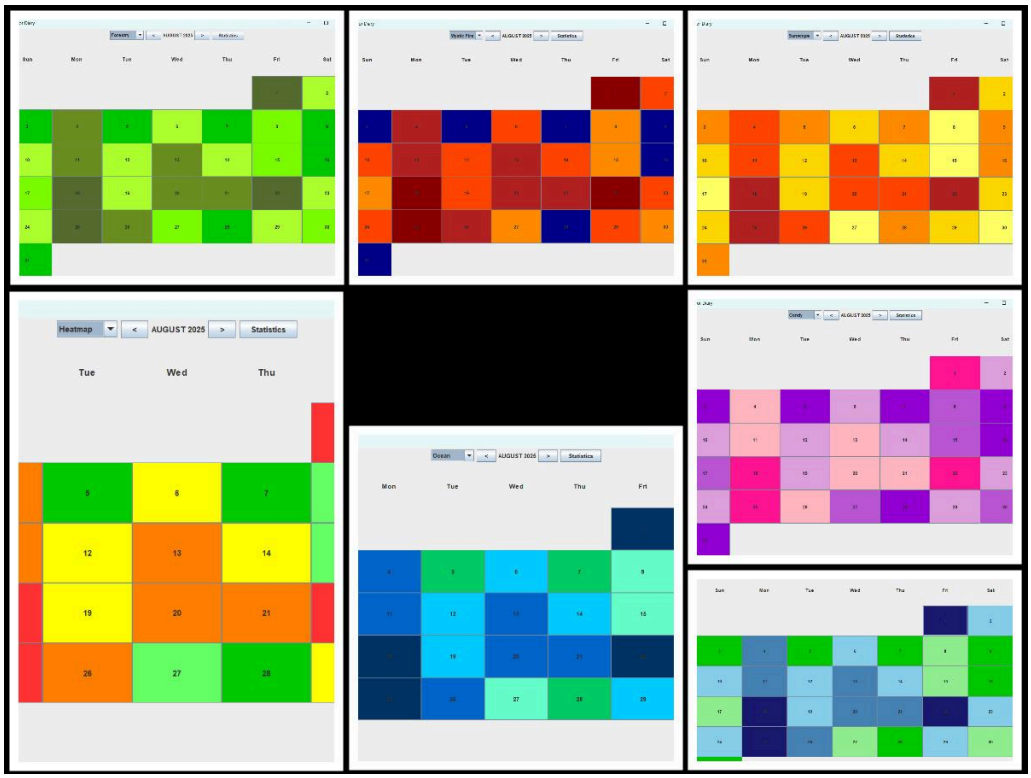
- Login screen – username and password

- Calendar view – with colored days

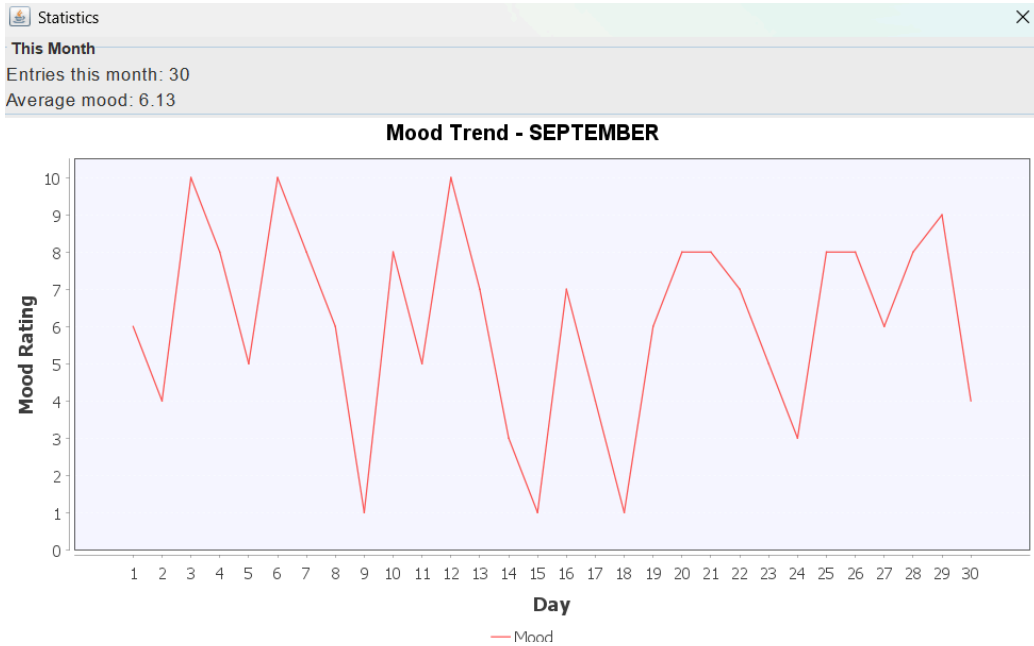


- Mood entry window – rating, emoji, and notes

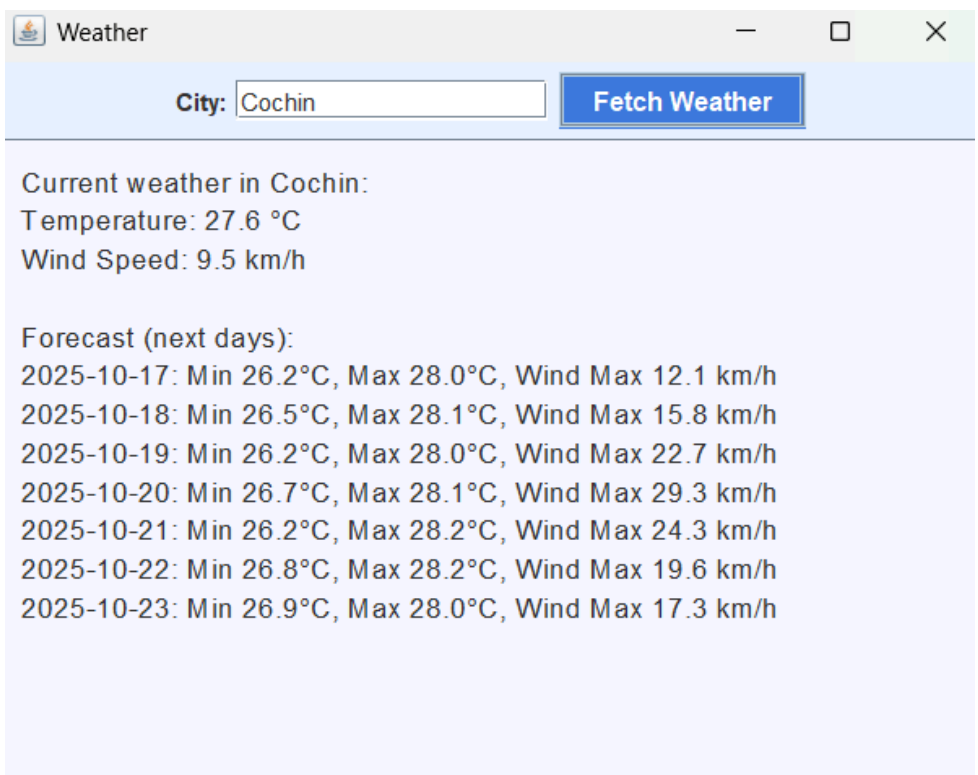




- Palettes Available



- Statistics chart – average mood line graph



- Weather window – temperature, wind speed, rainfall

CHAPTER 5
Conclusion

5.1 Summary of Work

The *Color Diary* project was developed as an emotional tracking system that allows users to record, visualize, and reflect on their daily moods. The project began with the idea of creating a more personalized version of a diary — one that could turn emotions into colors, making it easier for users to see patterns over time.

The final system consists of several well-integrated modules: a login screen for user authentication, a main calendar interface for daily entries, a statistics window to visualize monthly trends, and an optional weather window to display live environmental data. Each of these modules works together to give users a smooth, interactive experience.

The application's main feature is the **color-coded calendar**, where each day is shaded according to the user's recorded mood rating (1–10). This visual feedback helps users recognize long-term emotional patterns that are often hard to see in ordinary text diaries. A range of color palettes such as "Heatmap," "Ocean," and "Serenity" allows personalization and makes the interface more expressive.

5.2 Achievements and Observations

Over the course of implementation, several important outcomes were achieved:

1. **Functional Core System:**
   The calendar successfully retrieves and displays mood data for each date, saving new entries and updating existing ones. It is visually appealing and easy to navigate.

2. **Database Integration:**
   Initially, MySQL was used for data storage, but later the system was refactored to use **SQLite**, making it completely local and offline-capable. SQLite was chosen because it requires no additional setup and can be distributed with the application.

3. **Secure User Login:**
   The login screen ensures user-level privacy by encrypting passwords before saving them to the database. Each user's data is isolated, allowing multiple users to safely use the same system.

4. **Data Visualization:**
   The addition of **JFreeChart** brought analytical features such as line graphs showing mood variation across the month, along with numerical data like average mood and total entries.

5. **Weather Integration:**
   By connecting to the **Open-Meteo API**, the app now offers a weather window that can display the current temperature, wind speed, and short-term forecast. This adds context, since external weather conditions can affect a person's emotional state.

6. **User-Friendly Interface:**
   The project uses **Java Swing** for its graphical interface. Although Swing is an older toolkit, careful layout management and color design made the final interface look modern, clean, and intuitive.

7. **Portability and Maintainability:**
   The switch to SQLite and the modular structure of classes (such as MoodManager, DatabaseManager, StatsManager, and WeatherWindow) made the program portable and easier to maintain. The system is now independent of heavy external dependencies.

5.3 Limitations

While *Color Diary* achieved its main goals, a few limitations were observed during testing:

- The interface layout can appear slightly different across operating systems and display resolutions because Swing does not handle scaling uniformly.

- The weather window depends on an internet connection to fetch data from the Open-Meteo API.

- The emoji selection feature is basic and may not display perfectly on all systems due to Unicode rendering differences in Java.

- Currently, the application works for single users at a time. Multi-user data sharing or cloud synchronization is not yet implemented.

Despite these issues, none of them prevent the core functionality from working. They can be addressed in future updates.

5.4 Future Enhancements

Several ideas have been identified to expand *Color Diary* into a more complete and intelligent self-tracking tool:

1. **Personalized Mood Quotes:**
   The application could display motivational or reflective quotes each day based on the recorded mood. For example, positive and encouraging quotes for low moods, and philosophical or inspirational ones for higher moods.

2. **Event and Task Linking:**
   Integration with a basic to-do list or calendar event tracker could help users connect their mood changes with daily activities like exams, workdays, or holidays.

3. **Mood Aura Visualization:**
   Inspired by concepts like Spotify's "audio aura," the app could create blended color visualizations of average moods over time, forming a kind of emotional palette that evolves monthly.

4. **Theme and Dark Mode:**
   A modern dark mode and theme customization would make the interface more comfortable for long use, especially at night.

5. **Data Export and Backup:**
   Users could export their diary data in CSV or PDF format to share with mental health professionals or for long-term backup.

6. **AI-based Mood Prediction:**
   Future versions could integrate basic AI features to suggest journaling prompts or predict mood trends using previous entries.

7. **Improved Emoji and Media Support:**
   Expanding the emoji picker and allowing image or voice notes could make the diary more expressive and personal.

These enhancements would not only make *Color Diary* more useful but also more engaging as a daily companion.

5.5 Conclusion

In conclusion, *Color Diary* successfully demonstrates how technology can help visualize emotional wellbeing in a simple, personal way. The project combined basic programming, data visualization, and interface design skills into a cohesive and functional application. Through color-coded entries, statistical summaries, and simple integration of online data, the diary gives users both reflection and insight into their emotional lives.

The project also highlights how even small software tools can support mental health awareness and personal reflection. Beyond being a programming exercise, it represents a meaningful application of computer science to everyday human experience. The process of designing and building the system improved our understanding of both technical and psychological aspects of user interaction.

Overall, *Color Diary* is an easy-to-use, reliable, and expandable diary application. It successfully meets its intended goals and leaves room for continued improvement. With more advanced features in the future, it could grow into a full-fledged mood analytics platform that combines emotional awareness with data-driven insight.

IV
References

Oracle Java Documentation – https://docs.oracle.com/javase/
(For Java Swing, AWT, and JDBC implementation details.)

MySQL Official Documentation – https://dev.mysql.com/doc/
(Used for database connection setup and SQL queries.)

SQLite Official Documentation – https://www.sqlite.org/docs.html
(For understanding SQLite database integration for offline use.)

JFreeChart Developer Guide – https://www.jfree.org/jfreechart/
(For mood trend chart visualization and graph creation.)

GeeksforGeeks, "Java Swing Tutorial: Building GUI Applications in Java," https://www.geeksforgeeks.org/java-swing-tutorial/
(Used for reference while designing the calendar interface.)

Stack Overflow Discussions – https://stackoverflow.com/
(Referred for troubleshooting and improving project functionality.)

V
Appendix
Source code

entry_id	user_id	entry_date	mood_rating	emoji	text_entry	modified	created_at
1	1	2025-09-13	7		Finally made this app functional woo!	0	2025-09-13 18:52:19
5	1	2025-09-14	10		Palettes implemented ! App largely done.	0	2025-09-13 19:26:19
7	1	2025-09-12	2		Bit stressful with all the DSA programs to write in lab	0	2025-09-13 19:32:07
9	1	2025-09-11	3			0	2025-09-13 20:34:40
10	1	2025-09-15	4		Hi	0	2025-09-13 20:38:52
26	1	2025-09-09	6			0	2025-09-13 22:03:56
29	1	2025-09-07	10			0	2025-09-13 22:05:31
30	1	2025-09-06	9			0	2025-09-13 22:05:36
32	1	2025-09-05	8			0	2025-09-14 14:32:01
33	1	2025-09-04	7			0	2025-09-14 14:32:21
34	1	2025-09-03	9			0	2025-09-14 14:59:15
35	1	2025-09-02	7	?		0	2025-09-14 14:59:21
36	1	2025-09-01	9			0	2025-09-14 14:59:26
45	1	2025-08-27	7		Onam celebration. Everyone was dressed to the nines. Celebration itself was ok enough.	0	2025-09-14 15:22:20
46	1	2025-08-28	10		Woo! first day of vacation. Getting started on work for this app.	0	2025-09-14 15:23:04
47	1	2025-08-29	5		Some problems with mySQL. But we're chugging along.	0	2025-09-14 15:24:24
48	1	2025-08-30	8		Took a break. Read a book. Lazy afternoon.	0	2025-09-14 15:24:56
49	1	2025-08-31	9		Life is good.	0	2025-09-14 15:25:42

- SQL Mood_entries table

```
public static void main(String[] args) {
        JWindow splash = new JWindow();
        JLabel label = new JLabel("Color Diary",
SwingConstants.CENTER);
        label.setFont(new Font("Arial", Font.BOLD, 36));
        label.setForeground(Color.WHITE);

        JPanel panel = new JPanel(new BorderLayout());
        panel.setBackground(new Color(44, 150, 53));
        panel.add(label, BorderLayout.CENTER);

        splash.add(panel);
        splash.setSize(400, 200);
        splash.setLocationRelativeTo(null);
        splash.setVisible(true);

        new java.util.Timer().schedule(new
java.util.TimerTask() {
            @Override public void run() {
```

```
                splash.setVisible(false);
                splash.dispose();
                SwingUtilities.invokeLater(() -> new
LoginScreen());
            }
        }, 3000);
    }
```

- Source code snippet